pfi

# Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

# Random Forests

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Trees that agree do not average away much

### The variance of an average of B numbers with correlation rho

```
rho x sigma^2   +   (1 - rho) x sigma^2 / B
```

### rho = 1

Trees agree perfectly: averaging removes nothing, however many there are.

### rho = 0

Trees share nothing: a large enough average removes all of their noise.

**0.2581**

tree correlation, bagging

**100 of 100**

trees asking PAY_0 first

# How tree correlation works

| | |
|---|---|
| **The problem** | Averaging helps only as far as the trees disagree, so measure how much they agree |
| **Score** | Every tree scores the validation accounts: one column of outputs per tree |
| **Correlate** | Correlate every pair of columns and average the pairs, each pair once |
| **Read it** | Near 1: the trees say the same thing. Near 0: each adds something the others lack |
| **What comes out** | One number: the average pairwise correlation of the trees' outputs |

# How a random forest works

| | |
|---|---|
| **The problem** | Bagged trees all ask about the strongest column first, so they err alike |
| **Rows** | Each tree is fitted on its own bootstrap sample of the rows |
| **Columns** | At every split a tree may choose only among a random subset of the columns |
| **Grow and average** | Grow each tree deep, then average the trees' class-1 probabilities |
| **You control** | n_estimators, max_features (columns per split), and min_samples_leaf |

# A random subset of columns at every split

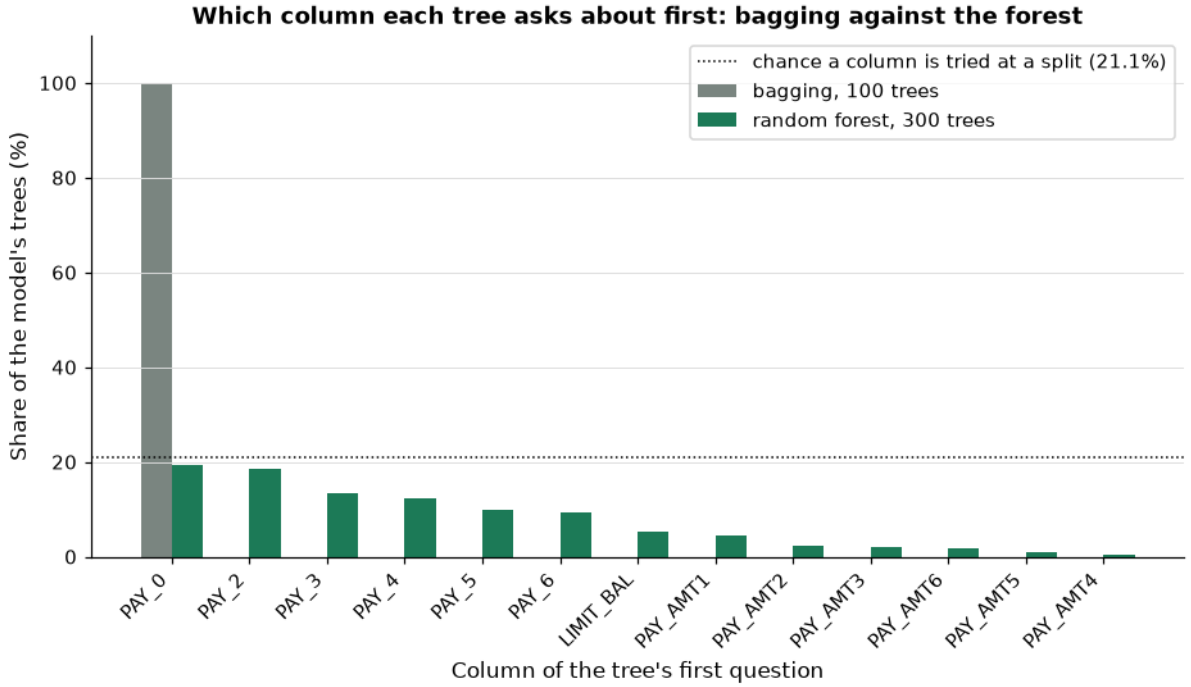
**4 of 19**

columns tried at each split ("sqrt")

**0.2105**

chance a given column is tried

PAY_0 comes first in 58 of 300 trees (19.3 percent), about as often as it is in the root's draw at all. The first questions use 13 different columns; bagging's 100 trees all start with PAY_0.




**A column a split cannot see cannot be chosen there: the trees differ from the first question.**

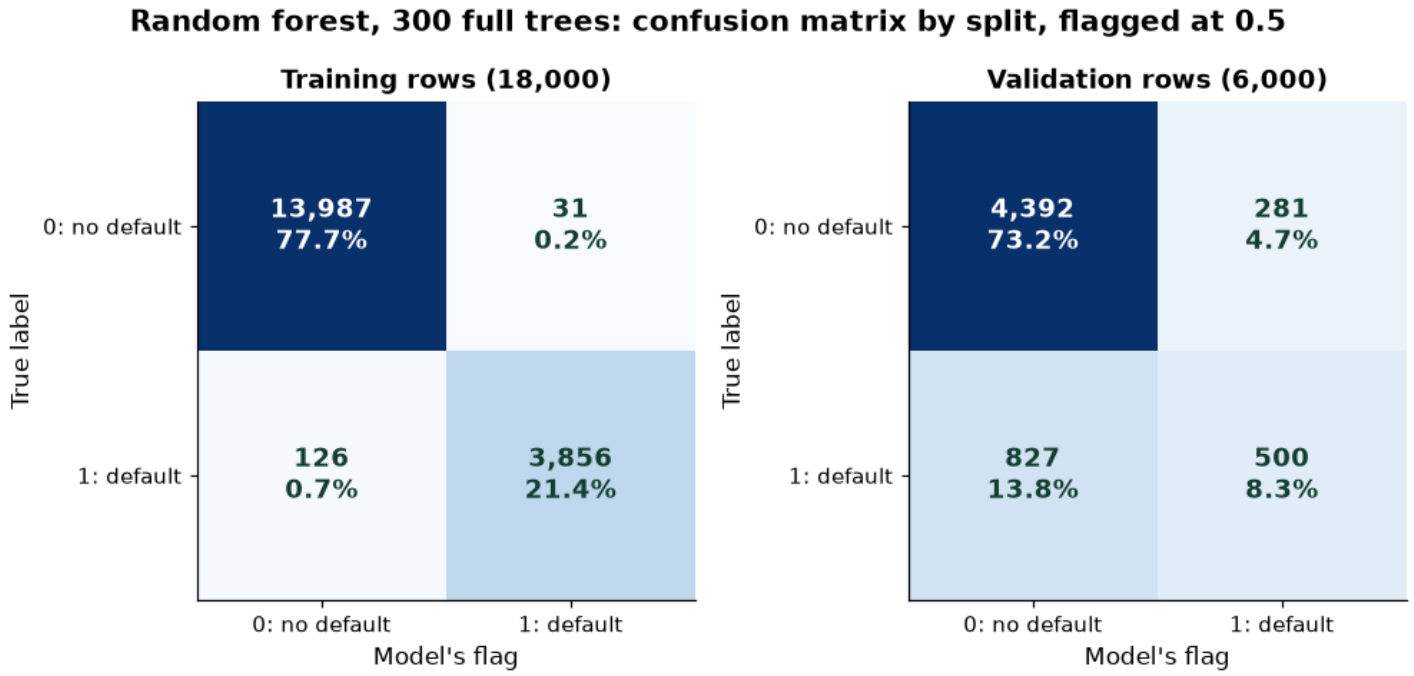
# The forest beside the bagged model: full trees

| Full trees | Validation ROC AUC | Average precision | Tree correlation |
|---|---|---|---|
| Bagging, 100 | 0.7602 | 0.5105 | 0.2581 |
| Random forest, 300 | 0.7600 | 0.5146 | 0.2487 |

**With full trees, the random columns change the first questions and barely change the agreement or the ranking.**

Full trees already disagree on many accounts, so there is little agreement left to remove.

# The standard view: one heatmap per split




**Training, then validation, on every card day. The test rows never get a panel.**

## How to read it

True label down the side, the model's flag at 0.5 across the top. Each cell: a count and its share of that split's rows.

## Full trees

Training panel: almost all on the diagonal. Validation panel: far from it (ROC AUC 0.7600). Bagging's panels look the same.

# The name on today's class

**Leo Breiman, October 2001**

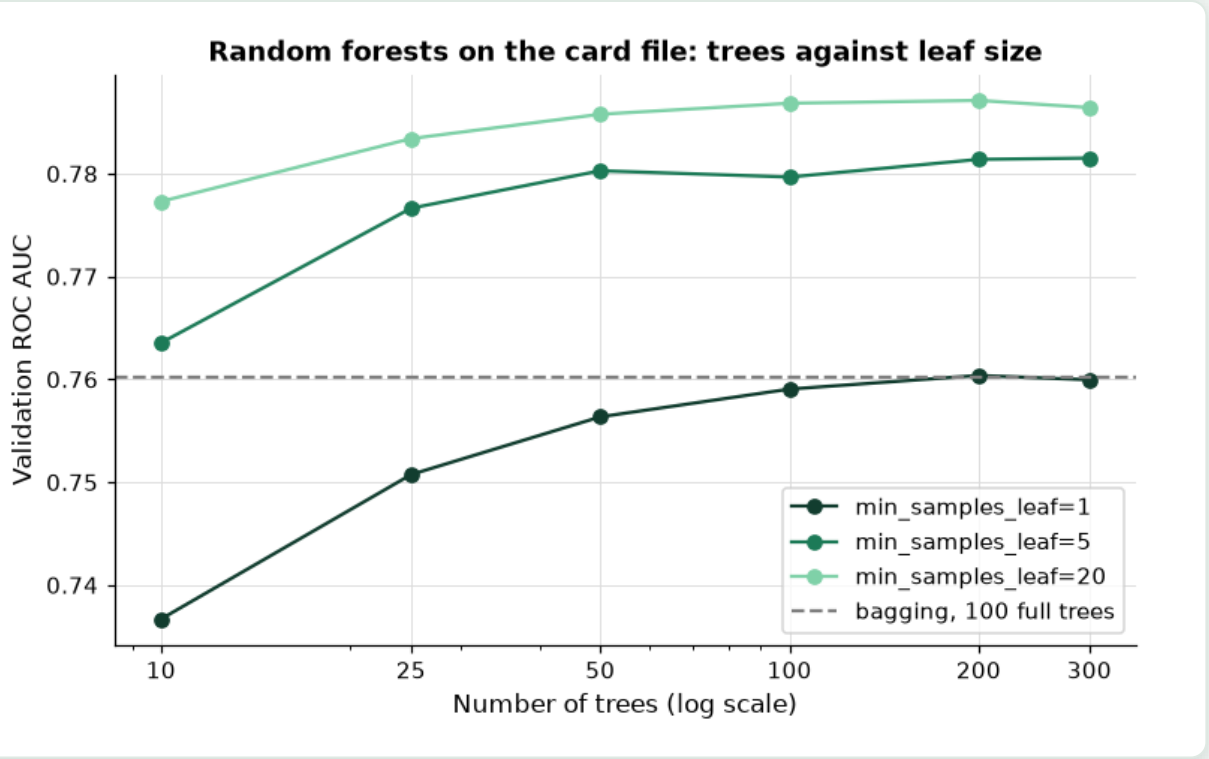
## "Random Forests"

Machine Learning, volume 45, issue 1, pages 5 to 32

**The same author as week 1's 1996 paper; the title is scikit-learn's class name.**

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1023/A:1010933404324). Bibliographic facts only; the paper itself was not read for this course.

# Leaf size matters more than tree count here

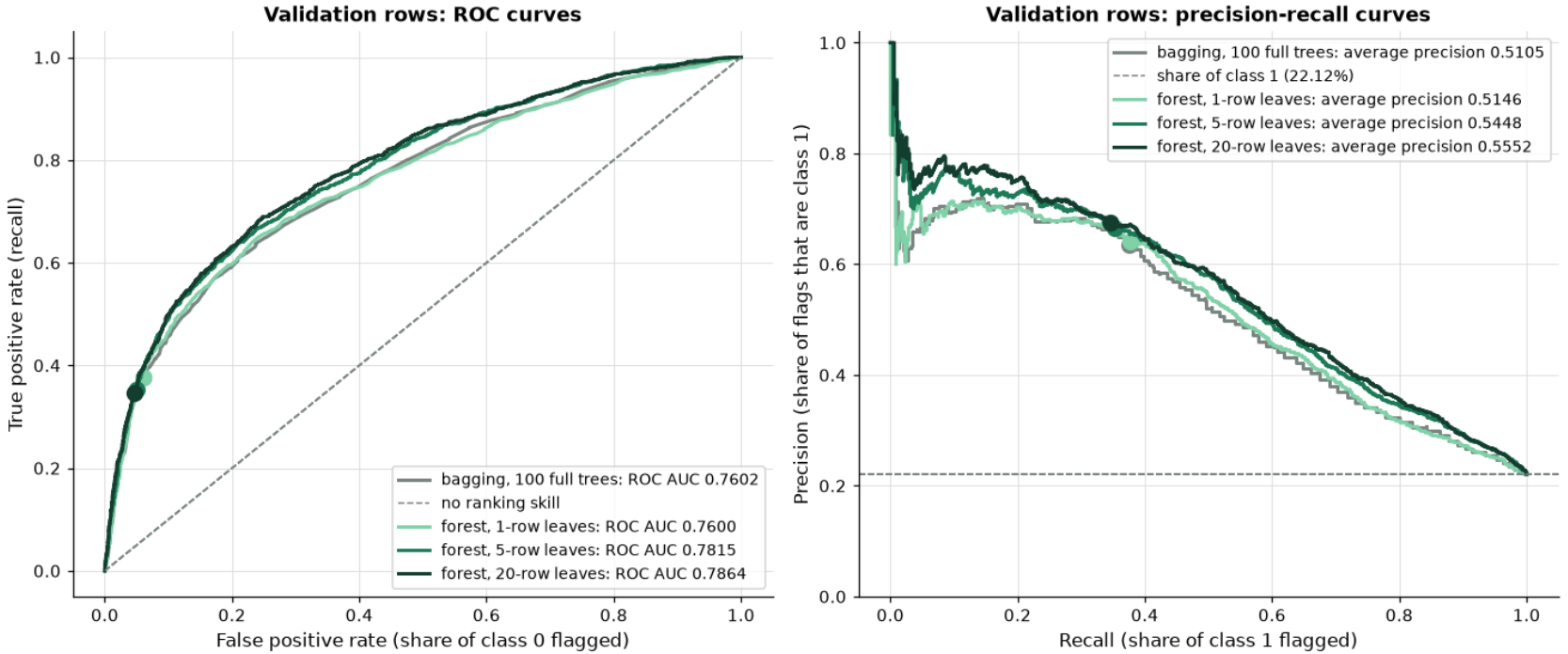



| Min leaf | One tree | Correlation | 300 trees |
|---|---|---|---|
| 1 | 0.6144 | 0.2487 | 0.7600 |
| 5 | 0.6801 | 0.4116 | 0.7815 |
| 20 | 0.7257 | 0.6311 | 0.7864 |

Bigger leaves: the trees agree more, each tree alone is much better, and the forest improves.

**An average is as good as its members and as independent as they are. On this file, better members win.**

# The curves by leaf size: validation rows




**Bigger leaves: higher curves, and the dot at 0.5 moves left as fewer accounts reach it.**

# max_features=None is bagging

| 100 full trees | Columns per split | Validation ROC AUC | Tree correlation |
|---|---|---|---|
| RandomForestClassifier, max_features=None | 19 | 0.7595 | 0.2577 |
| BaggingClassifier | 19 | 0.7602 | 0.2581 |

**Equal settings, nearly equal numbers: every tree asks PAY_0 first. Today's AI check.**

# How class weights work

| | |
|---|---|
| **The problem** | Defaults are a small share, so impurity splits attend mostly to the other accounts |
| **Weight** | A class weight counts each row of a class as more or less than one row |
| **balanced** | Each class gets weight n / (2 x its count); balanced_subsample redoes it per bootstrap sample |
| **You control** | The choice of weights |
| **What comes out** | A similar ranking at a higher level: the model's score, not a probability of default |

# Class weights: the same ranking, a higher level

**2.2602**

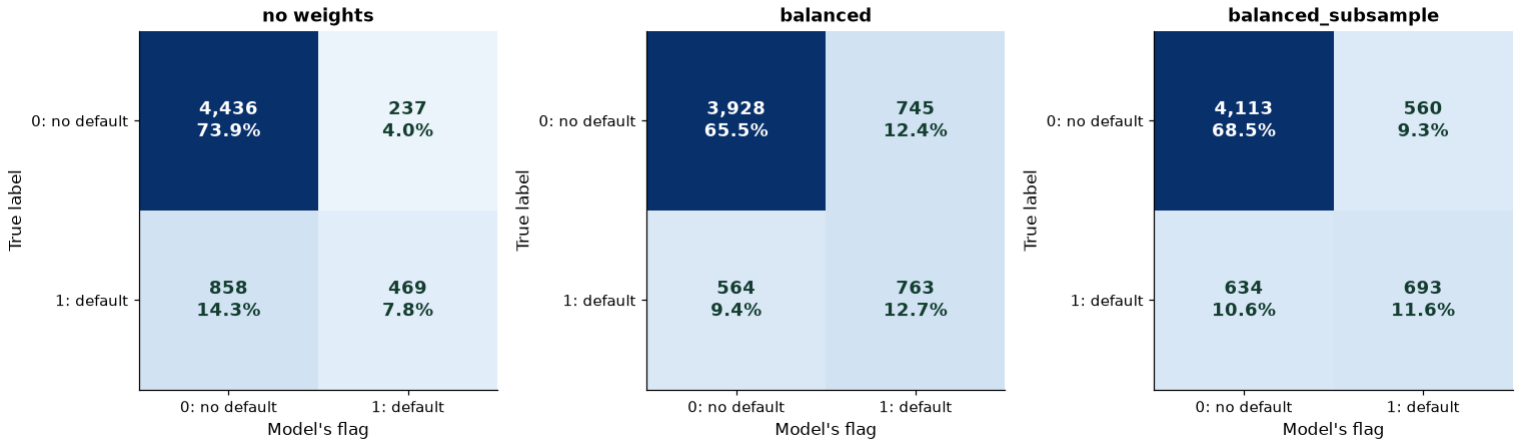
weight of each default

**0.7806**

ROC AUC, balanced

**0.3735**

mean output, balanced

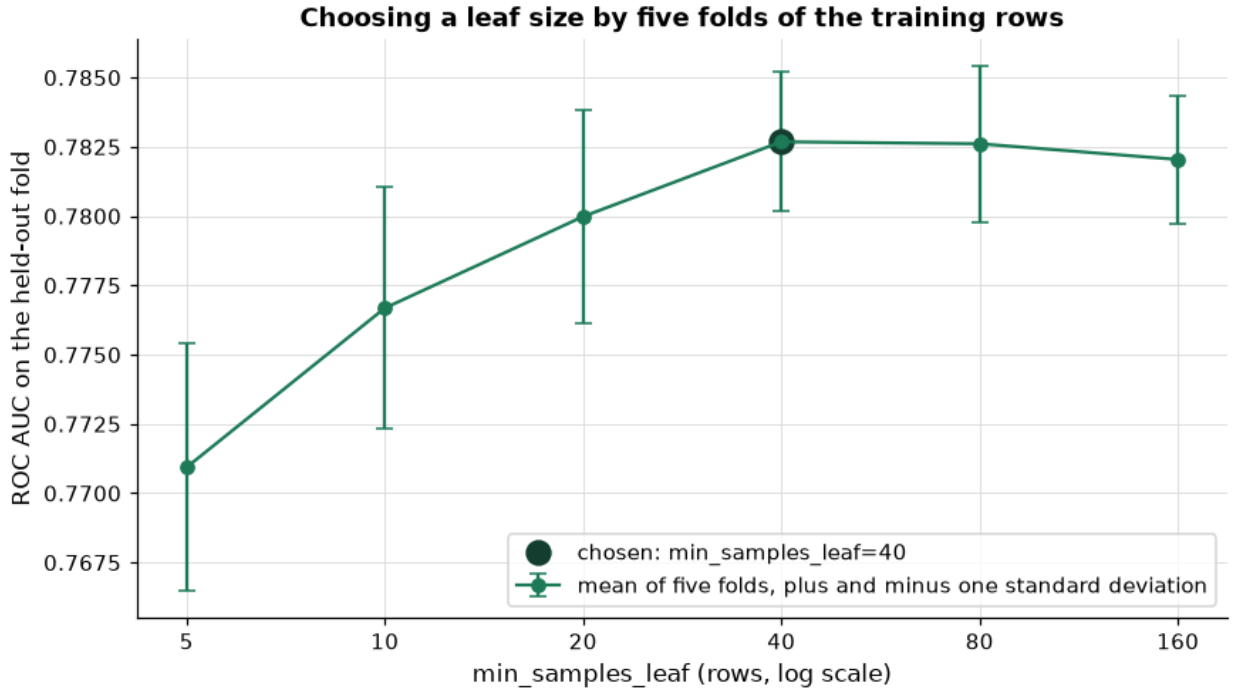



**Unweighted: ROC AUC 0.7815, mean output 0.2223, default rate 0.2212. A weighted forest's output is the model's score.**

# How a cross-validated search works

| | |
|---|---|
| **The problem** | A setting cannot be chosen on rows the forest was fitted on, or on the validation rows |
| **Folds** | Cut the training rows into five; fit on four, score on the fifth, five times |
| **Every candidate** | GridSearchCV does this for each leaf size and averages the five scores |
| **You control** | The candidates, the seeded folds, and the metric (ROC AUC) |
| **What comes out** | Mean fold scores, the best leaf size, and a forest refitted on all the training rows |

# Choosing a leaf size on the training rows




**40**

rows per leaf, chosen

**0.7844**

validation ROC AUC, reported once

50-tree forests, five folds of the training rows. Fold mean of the chosen leaf size: 0.7827, the best of six noisy numbers.

**The validation rows look and report. They never choose.**

# How extra trees work

| | |
|---|---|
| **The problem** | A forest still searches every threshold, and that search is where a tree fits its sample closely |
| **Random thresholds** | Draw a few columns, draw one threshold per column at random, keep the best random question |
| **Rows** | By default each tree sees all the training rows: no bootstrap |
| **Trade** | Each tree is a little weaker than a forest's: its thresholds are not the best ones |
| **You control** | The same settings as a forest; the output is again an average |

# Every setting of RandomForestClassifier

| Setting | Default | Tune first? |
|---|---|---|
| n_estimators | 100 | |
| max_depth | None | |
| min_samples_leaf | 1 | tune first |
| max_features | 'sqrt' | tune first |
| bootstrap | True | |
| max_samples | None | tune first |
| class_weight | None | |
| oob_score | False | |
| criterion | 'gini' | |
| ccp_alpha | 0.0 | |

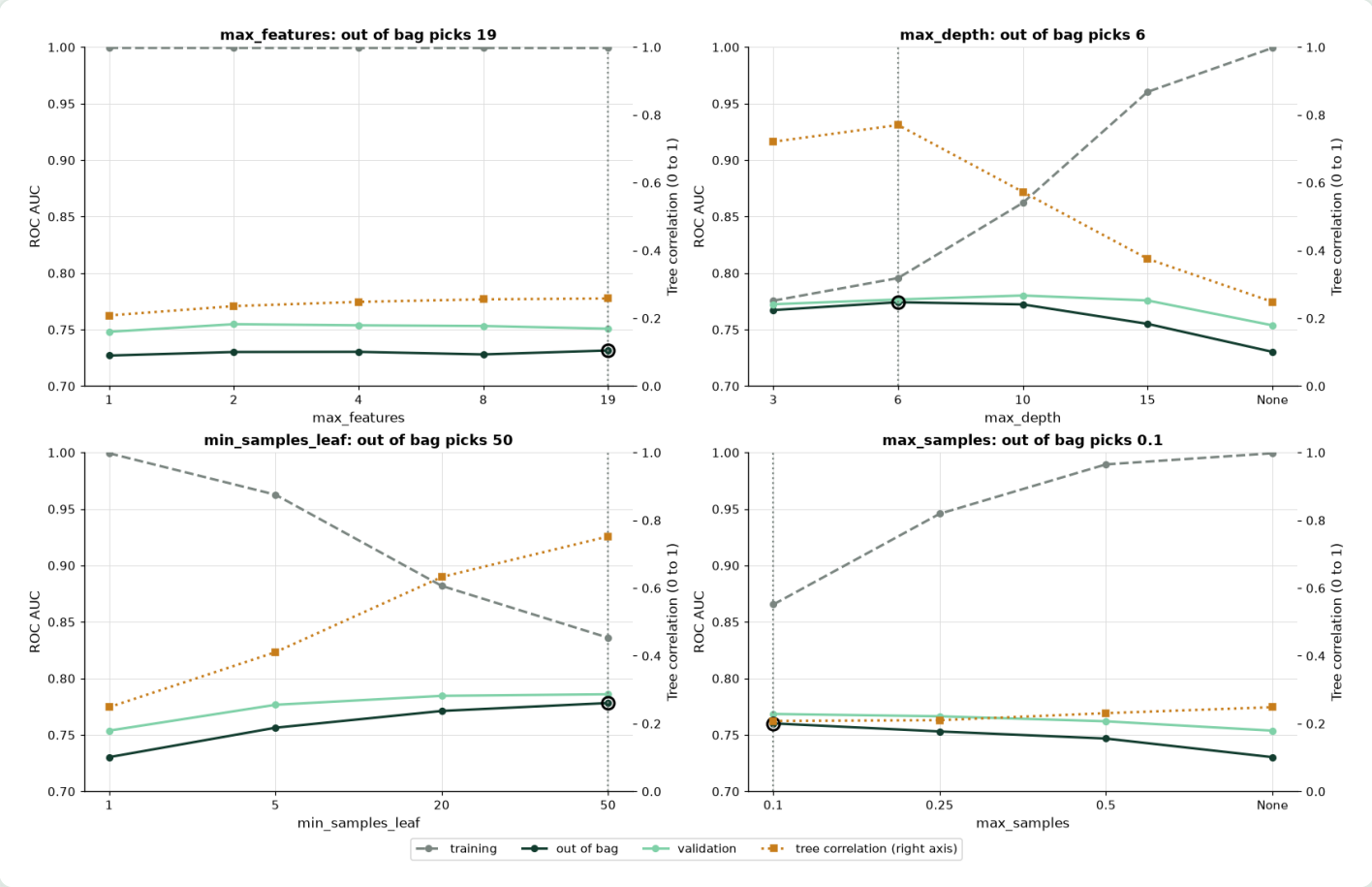
### Nineteen settings, by group

Tree shape (max_depth, min_samples_leaf), randomness (max_features, bootstrap, max_samples, seed), regularization, class weights, and housekeeping. Ten of the nineteen are shown.

### The three to tune first

min_samples_leaf, max_features, and max_samples: they decide how strong the trees are and how much they agree. max_depth does the leaf-size job from the other side.

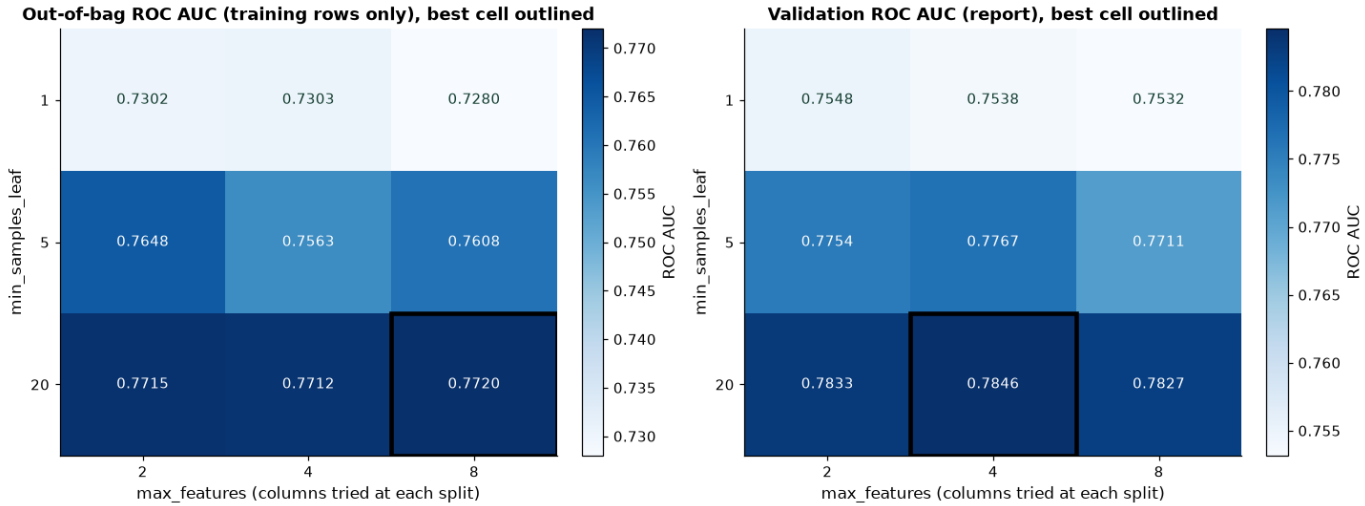
# Four settings, one at a time, chosen out of bag




| Setting | Pick | OOB | Valid. | Corr. |
|---|---|---|---|---|
| max_features | 19 | 0.7315 | 0.7508 | 0.2584 |
| max_depth | 6 | 0.7742 | 0.7766 | 0.7703 |
| min_samples_leaf | 50 | 0.7782 | 0.7860 | 0.7523 |
| max_samples | 0.1 | 0.7602 | 0.7685 | 0.2078 |

Dotted orange: how much the trees agree. min_samples_leaf moves the ranking; max_features moves the agreement.

# Two settings together, and extra trees




| Extra trees, 100 | ROC AUC | AP |
|---|---|---|
| bootstrap=False (default) | 0.7792 | 0.5473 |
| bootstrap=True | 0.7818 | 0.5509 |

## One default differs

ExtraTreesClassifier takes a forest's settings. Only bootstrap differs. The random thresholds are not a setting.

**Leaf size is the setting to tune first on this file; columns per split stays at its default unless there is a reason.**

# Excel side by side

### Excel side by side (tested in Excel)

| | |
|---|---|
| `=INT(SQRT(19))` | 4 columns per split, "sqrt" |
| `=INT(0.5*19)` | 9 columns per split, max_features=0.5 |
| `=1-COMBIN(18,4)/COMBIN(19,4)` | 0.2105, a given column tried |
| `=CORREL(B2:B6001,C2:C6001)` | bagging 0.2564, forest 0.2362 |
| `ten CORREL pairs, then =AVERAGE(H1:H10)` | 0.2455, five trees |
| `=ROWS(A2:A18001)/(2*COUNTIF(A2:A18001,1))` | 2.2602, weight of a default |
| `=AVERAGE(C1:C5) and =STDEV.P(C1:C5)` | the search's mean and SD |

**No forest and no tree in Excel. The arithmetic around one, typed into Excel by script and checked against Python.**

# Save the day: one file's history

```
git log --oneline -- results.csv
```

```
Experiment log: leaf sizes, class weights, the searched forest, extra trees
Experiment log: bagging and a random forest of full trees, validation rows
```

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
bagging_100_full_trees,validation,0.7602,0.5105,0.5000,0.8143,0.6357,0.3760,0.4725
random_forest_300_full_trees,validation,0.7600,0.5146,0.5000,0.8153,0.6402,0.3768,0.4744
random_forest_300_leaf_5,validation,0.7815,0.5448,0.5000,0.8175,0.6643,0.3534,0.4614
random_forest_300_leaf_20,validation,0.7864,0.5552,0.5000,0.8185,0.6750,0.3459,0.4574
random_forest_300_leaf_5_balanced,validation,0.7806,0.5386,0.5000,0.7818,0.5060,0.5750,0.5383
random_forest_50_leaf_40_searched,validation,0.7844,0.5534,0.5000,0.8198,0.6814,0.3482,0.4608
extra_trees_300_leaf_5,validation,0.7814,0.5490,0.5000,0.8202,0.6782,0.3557,0.4666
```

**Two commits touched results.csv; the code commit did not. A path after -- gives that file's history alone.**

# The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

## Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

## Code back

Written for the course in the style of an AI answer. Read it.

→

3

## Test first

Assert what the docstring promises. Run it: it fails.

→

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: a "random forest" with max_features=None: bagging under another class name.**

*Code that runs is not the same as code that is right.*

# What you can do now

- Measure how much an ensemble's trees agree
- Work out the columns a split tries
- max_features=None is bagging
- Leaf size moves a forest more than tree count
- Class weights: same ranking, the model's score

**Week 1, Day 4** Impurity and permutation importance